

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
AI and Application Development and Jira

AI MINI PROJECT

AI Disaster Early Warning System

Student Name	Aashu Yadav
Roll No.	T050
Class / Batch	TYIT / 1
Subject	AI and Application Development and Jira
Academic Year	2026 - 2027

Submitted to: Prof. Chetana Tanavade

Introduction:

The AI Disaster Early Warning System is a Flask-based web application that uses Machine Learning to predict environmental disaster risk.

The system accepts environmental information such as rainfall, water level, and wind speed from the user. It uses a trained Decision Tree Classifier to classify the conditions into Low, Medium, or High Risk.

The project is useful because it provides a simple way to monitor environmental conditions and generate an early warning based on the entered data.

Problem Statement:

To develop an AI-based web application that analyzes rainfall, water level, and wind speed and predicts the level of disaster risk as Low, Medium, or High, along with suitable warning messages and safety instructions.

Objectives:

- To create a sample dataset containing environmental disaster parameters.
- To use rainfall, water level, and wind speed as input features.
- To apply a Decision Tree Machine Learning algorithm.
- To train the model for disaster-risk classification.
- To provide risk predictions and warning messages through a Flask web application.

Technologies Used:

Programming Language : **Python**

Framework : **Flask**

Libraries : **Pandas & Scikit-learn**

Front-End : **HTML & CSS**

Development Tools : **VS Code / Python & Web Browser**

Dataset:

Source : The dataset is created directly within the project code in model.py. It is not an externally downloaded

Dataset Name : Sample Disaster Dataset

Number of Records : 15 records

Features :

The dataset contains three input features:

1. Rainfall
2. Water Level
3. Wind Speed

Target Variable : Risk

The target contains three categories : Low, Medium, & High.

Dataset Attributes:

Attribute	Description
Rainfall	Rainfall measurement
Water_Level	Water level measurement

Wind_Speed	Wind speed measurement
Risk	Predicted risk category

Dataset Distribution:

Risk Category	Records
---------------	---------

Low	5
Medium	5

High	5
Total	15

Methodology / Working:

The project follows these steps:

1. A sample environmental dataset is created.
2. The dataset is converted into a Pandas DataFrame.
3. Rainfall, water level, and wind speed are selected as input features.
4. Risk is selected as the target variable.
5. A Decision Tree Classifier is created.
6. The model is trained using the dataset.
7. The Flask web application accepts user input.
8. The entered values are passed to the trained Machine Learning model.
9. The model predicts the risk level.
10. The application displays the prediction along with a warning message and safety instructions.

Working Flow:

User Input → Flask Application → Decision Tree Model → Risk Prediction → Warning Message → Safety Instructions

Algorithm Used:

- **Decision Tree Classifier** : The project uses the Decision Tree Classifier from Scikit-learn.
- **What is a Decision Tree?**
A Decision Tree is a supervised Machine Learning algorithm used for classification. It makes decisions using a tree-like structure based on input features.
- **Why is it used?**
It is used in this project to classify environmental conditions into: Low Risk, Medium Risk, High Risk
- **How does it work?**
The model learns the relationship between:
Rainfall + Water Level + Wind Speed → Risk
The trained model receives new environmental values and predicts the corresponding risk category.
- **The project uses:**
model = DecisionTreeClassifier(random_state=42) and trains it using : model.fit(X, y)

Implementation:

Step 1: Import Libraries

import pandas as pd

from sklearn.tree import DecisionTreeClassifier

Pandas is used for handling the dataset, while Scikit-learn provides the Decision Tree Classifier.

Step 2: Create Dataset

The project creates a dataset containing:

- Rainfall
- Water_Level
- Wind_Speed

Risk : There are 15 records in total.

Step 3: Select Features

```
X = df[["Rainfall", "Water_Level", "Wind_Speed"]]
```

The three environmental parameters are used as input features.

Step 4: Select Target

```
y = df["Risk"]
```

The Risk column is used as the target variable.

Step 5: Train Model

- `model = DecisionTreeClassifier(random_state=42)`

```
model.fit(X, y)
```

The Decision Tree learns from the sample dataset.

Step 6: Prediction

- `def predict_risk(rainfall, water_level, wind_speed):`
- `new_data = [[rainfall, water_level, wind_speed]]`
- `prediction = model.predict(new_data)[0]`

```
return prediction
```

This function accepts new environmental values and returns the predicted risk.

Step 7: Flask Web Application

The Flask application receives:

- Location
- Rainfall
- Water Level
- Wind Speed

The /predict route processes the submitted data and calls the Machine Learning prediction function.

Step 8: Display Warning

The application displays different messages depending on the prediction:

- **High Risk (Possible disaster risk detected. Take safety precautions immediately).**
- **Medium Risk (Stay alert and monitor the situation).**
- **Low Risk (No immediate high risk detected).**

The application also displays safety instructions such as staying updated with official alerts, keeping emergency supplies ready, avoiding unsafe/flooded areas, and following local authorities.

Screenshots to Include

For your documentation, add screenshots of:

1. model.py:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

# Sample disaster dataset
data = {
    "Rainfall": [
        10, 15, 20, 25, 30,
        40, 50, 60, 70, 80,
        90, 100, 110, 120, 130
    ],
    "Water_Level": [
        10, 12, 15, 18, 20,
        25, 30, 35, 40, 45,
```

```

50, 55, 60, 70, 80
],

"Wind_Speed": [
5, 6, 8, 10, 12,
15, 18, 20, 22, 25,
28, 30, 32, 35, 40
],

"Risk": [
"Low", "Low", "Low", "Low", "Low",
"Medium", "Medium", "Medium", "Medium", "Medium",
"High", "High", "High", "High", "High"
]
}

# Create DataFrame
df = pd.DataFrame(data)

# Features
X = df[["Rainfall", "Water_Level", "Wind_Speed"]]

# Target
y = df["Risk"]

# Create Decision Tree
model = DecisionTreeClassifier(random_state=42)

# Train model
model.fit(X, y)

print("AI Model Trained Successfully!")

def predict_risk(rainfall, water_level, wind_speed):
    new_data = [[rainfall, water_level, wind_speed]]
    prediction = model.predict(new_data)[0]
    return prediction

```

2. app.py:

```

from flask import Flask, render_template, request
from model import predict_risk

```

```

app = Flask(__name__)
@app.route("/")
def home():
    return render_template("index.html")

```

```

@app.route("/predict", methods=["POST"])
def predict():

```

```

    location = request.form["location"]
    rainfall = float(request.form["rainfall"])
    water_level = float(request.form["water_level"])
    wind_speed = float(request.form["wind_speed"])

```

```

# AI Prediction
prediction = predict_risk(
    rainfall,
    water_level,
    wind_speed
)

# Warning message
if prediction == "High":

    message = "Possible disaster risk detected. Take safety precautions immediately."

    warning = "Next 30 minutes may be critical."

elif prediction == "Medium":

    message = "Stay alert and monitor the situation."

    warning = "Keep emergency items ready."

else:

    message = "No immediate high risk detected."

    warning = "Continue normal monitoring."

    return render_template(
        "index.html",
        location=location,
        rainfall=rainfall,
        water_level=water_level,
        wind_speed=wind_speed,
        prediction=prediction,
        message=message,
        warning=warning
    )
if __name__ == "__main__":
    app.run(debug=True)

```

3. Dataset/model code :

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier
# Dataset
data = {
    "Rainfall": [
        10, 15, 20, 25, 30,
        40, 50, 60, 70, 80,
        90, 100, 110, 120, 130
    ],
    "Water_Level": [
        10, 12, 15, 18, 20,
        25, 30, 35, 40, 45,
        50, 55, 60, 70, 80
    ]
}

```

```

    ],

    "Wind_Speed": [
    5, 6, 8, 10, 12,
    15, 18, 20, 22, 25,
    28, 30, 32, 35, 40
    ],

    "Risk": [
    "Low", "Low", "Low", "Low", "Low",
    "Medium", "Medium", "Medium", "Medium", "Medium",
    "High", "High", "High", "High", "High"
    ]
}

# Convert dataset into DataFrame
df = pd.DataFrame(data)

# Input features
X = df[["Rainfall", "Water_Level", "Wind_Speed"]]

# Target variable
y = df["Risk"]

# Create Decision Tree model
model = DecisionTreeClassifier(random_state=42)

# Train the model
model.fit(X, y)

print("AI Model Trained Successfully!")

# Prediction function
def predict_risk(rainfall, water_level, wind_speed):

    new_data = [[rainfall, water_level, wind_speed]]

    prediction = model.predict(new_data)[0]

    return prediction

```

4. Input form :

AI Disaster Early Warning System
AI-based Environmental Risk Monitoring System

Enter Environmental Data

📍 Location

🌧️ Rainfall (mm)

💧 Water Level

💨 Wind Speed (km/h)

CHECK RISK

AI-based Environmental Disaster Early Warning System
 © 2024 All Rights Reserved

5. Low Risk output :

Enter Environmental Data

📍 Location

🌧️ Rainfall (mm)

💧 Water Level

💨 Wind Speed (km/h)

CHECK RISK

Prediction Result

Location: andheri
 Rainfall: 20.0 mm
 Water Level: 10.0
 Wind Speed: 10.0 km/h

🟢 LOW RISK
 No immediate high-risk detected.
 Continue normal monitoring.

📋 Safety Instructions

- Stay updated with official alerts.
- Keep emergency supplies ready.
- Avoid unsafe or flooded areas.
- Follow instructions from local authorities.

6. Medium Risk output :

Enter Environmental Data

📍 Location

kolkata

🌧️ Rainfall (mm)

15

💧 Water Level

50

💨 Wind Speed (km/h)

20

CHECK RISK

Prediction Result

Location: kolkata
Rainfall: 15.0 mm
Water Level: 50.0
Wind Speed: 20.0 km/h

🚨 MEDIUM RISK

Stay alert and monitor the situation.
Keep emergency items ready.

📋 Safety Instructions

- Stay updated with official alerts.
- Keep emergency supplies ready.
- Avoid unsafe or flooded areas.
- Follow instructions from local authorities.

7. High Risk output :

Enter Environmental Data

📍 Location

🌧️ Rainfall (mm)

💧 Water Level

💨 Wind Speed (km/h)

CHECK RISK

Prediction Result

Location: Mumbai
 Rainfall: 95.0 mm
 Water Level: 52.0
 Wind Speed: 29.0 km/h

🚨 **HIGH RISK**

Possible disaster risk detected. Take safety precautions immediately.
Next 30 minutes may be critical.

📋 **Safety Instructions**

- Stay updated with official alerts.
- Keep emergency supplies ready.
- Avoid unsafe or flooded areas.
- Follow instructions from local authorities.

Results:

The developed Flask application successfully accepts environmental information and generates a disaster-risk classification.

Input

The user enters:

- Location
- Rainfall in mm
- Water Level
- Wind Speed in km/h

Output:

The system predicts one of three categories:

Prediction	Output
Low	LOW RISK

Medium	MEDIUM RISK
High	HIGH RISK

Example:

For higher environmental values such as:

- Rainfall: 100
- Water Level: 55
- Wind Speed: 30

the model classifies the condition as: **HIGH RISK**

The application then displays the corresponding warning message and safety instructions.

Performance:

The current project does not calculate model accuracy, precision, recall, F1-score, or a confusion matrix. Therefore, an accuracy percentage should not be added to the documentation unless you separately implement model evaluation.

Advantages:

- Provides quick environmental risk classification.
- Uses Machine Learning for prediction.
- Simple and user-friendly web interface.
- Provides different warning messages for different risk levels.
- Provides safety instructions to the user.

Limitations:

- The dataset contains only 15 records.
- The dataset is a sample dataset created within the project.
- The system does not use real-time weather or sensor data.
- The current project does not perform separate model accuracy testing.

Applications:

The system can be used as a prototype for:

- Disaster-risk monitoring
- Flood-risk awareness
- Environmental monitoring
- Disaster management applications
- Community safety systems
- Educational AI/ML projects
- Future IoT-based disaster monitoring systems

Conclusion:

The AI Disaster Early Warning System was successfully developed as a Flask-based Machine Learning web application.

The system uses rainfall, water level, and wind speed as input parameters and applies a Decision Tree Classifier to predict whether the environmental condition represents Low, Medium, or High Risk.

The project demonstrates the integration of Artificial Intelligence, Machine Learning, Python, and Flask to create a simple disaster-risk monitoring and early-warning application.

Future Scope:

The project can be improved by:

- Using a larger real-world disaster dataset.
- Integrating real-time weather data.
- Connecting IoT sensors for automatic data collection.
- Adding a database for storing environmental records.
- Adding model accuracy and performance evaluation.
- Comparing different Machine Learning algorithms.
- Adding graphical data visualization.
- Adding SMS/email/mobile emergency notifications.
- Developing a mobile application.
- Integrating real-time location-based monitoring.

References:

Video Link: <https://drive.google.com/file/d/1aAe5OiQvGwKZ3qkBxv8FB0hev11we0WY/view?usp=sharing>

GitHub Link: <https://github.com/Aashu-yadav17/Ai-practical/tree/main/project>